

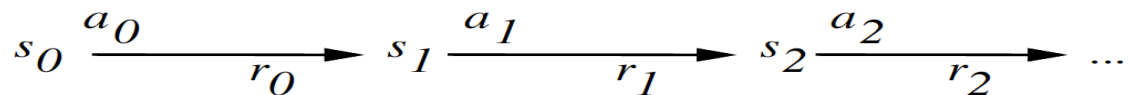
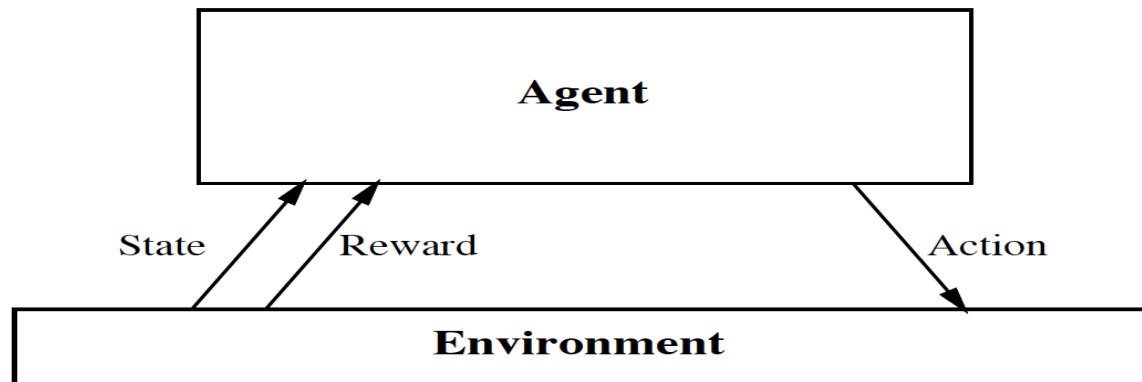
Presentation Outline

- Introduction
- RL Basics
- An Example of RL
- Conclusion

Reinforcement Learning

Reinforcement learning is an area of machine learning concerned with how intelligent agents ought to take actions in an environment in order to maximize the notion of cumulative reward.

Reinforcement learning is one of three basic machine learning paradigms, alongside supervised learning and unsupervised learning.



Your action influences the state of the world which determines its reward

Introduction

- Supervised Learning: Immediate feedback (labels provided for every input).
(classification and regression)
 - Unsupervised Learning: No feedback (no labels provided).
(clustering, dimensionality reduction)
 - Reinforcement Learning: Delayed scalar feedback (a number called reward).
(Autonomic management and control)
 - RL deals with agents that must sense & act upon their environment.
This is combines classical AI and machine learning techniques.
 - Examples:
 - Robot-soccer
 - How to invest in shares
 - Modeling the economy through rational agents
 - Learning how to fly a helicopter
- <https://www.youtube.com/watch?v=VCdxqn0fcnE>
- AlphaGo: <https://www.youtube.com/watch?v=-D3MyAm3JFo>

Complications

- The outcome of your actions may be uncertain
- You may not be able to perfectly sense the state of the world
- The reward may be stochastic.
- Reward is delayed (i.e. finding food in a maze)
- You may have no clue (model) about how the world responds to your actions.
- You may have no clue (model) of how rewards are being paid off.
- The world may change while you try to learn it
- How much time do you need to explore uncharted territory before you exploit what you have learned?

Markov Decision Process

A **Markov decision process (MDP)** is a discrete time stochastic control process.

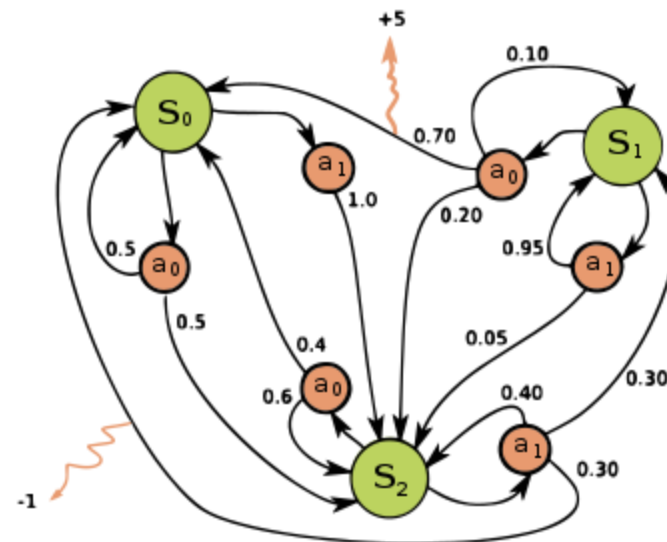
It provides a mathematical framework for modeling decision making in situations where outcomes are partly random and partly under the control of a decision maker.

MDPs are useful for studying optimization problems solved via dynamic programming and reinforcement learning.

MDP

A Markov decision process is a 4-tuple (S, A, P_a, R_a) , where

- S is a finite set of states,
- A is a finite set of actions (alternatively, A_s is the finite set of actions available from state s),
- $P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$ is the probability that action a in state s at time t will lead to state s' at time $t + 1$,
- $R_a(s, s')$ is the immediate reward (or expected immediate reward) received after transitioning from state s to state s' , due to action a



The Objective

- To learn an optimal *policy* that maps states of the world to actions of the agent.
I.e., if this patch of room is dirty, I clean it. If my battery is empty, I recharge it.

$$\pi : \mathcal{S} \rightarrow \mathcal{A}$$

- What is it that the agent tries to optimize?
Answer: the **total future discounted reward**:

$$\begin{aligned} V^\pi(s_t) &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\ &= \sum_{i=0}^{\infty} \gamma^i r_{t+i} \quad 0 \leq \gamma < 1 \end{aligned}$$

Note: immediate reward is worth more than future reward.

Policy from Value Function

- Let's say we have access to the optimal value function that computes the total future discounted reward $V^*(s)$
- What would be the optimal policy $\pi^*(s)$?
- Answer: we choose the action that maximizes:

$$p^*(s) = \underset{a}{\operatorname{argmax}} \left(r(s,a) + \gamma V^*(d(s,a)) \right)$$

- We assume that we know what the reward will be if we perform action “a” in state “s”:
- We also assume we know what the next state of the world will be if we perform action “a” in state “s”:

Q-Function

- One approach to RL is to try to estimate $V^*(s)$.

$$V^*(s) = \max_a [r(s,a) + \gamma V^*(s',a)] \quad \text{Bellman Equation:}$$

- However, this approach requires you to know $r(s,a)$ and $\delta(s,a)$.
- This is unrealistic in many real problems. What is the reward if a robot is exploring mars and decides to take a right turn?
- Fortunately we can circumvent this problem by exploring and experiencing how the world reacts to our actions. We need to *learn* r & δ .
- We want a function that directly learns good state-action pairs, i.e. what action should I take in this state. We call this $Q(s,a)$.
- Given $Q(s,a)$ it is now trivial to execute the optimal policy, *without knowing* $r(s,a)$ and $\delta(s,a)$. We have:

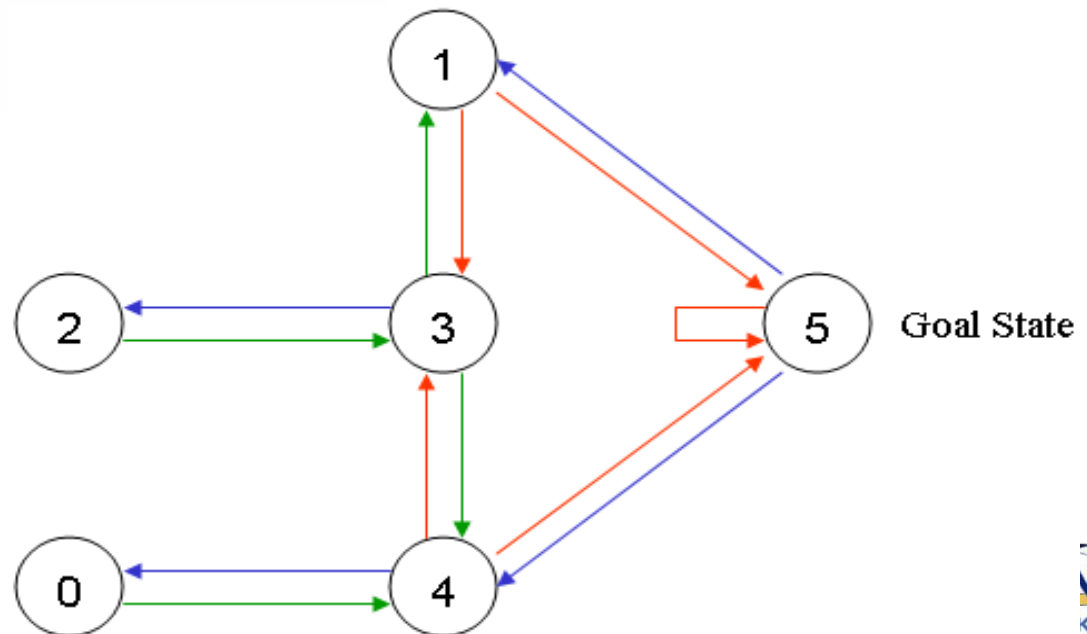
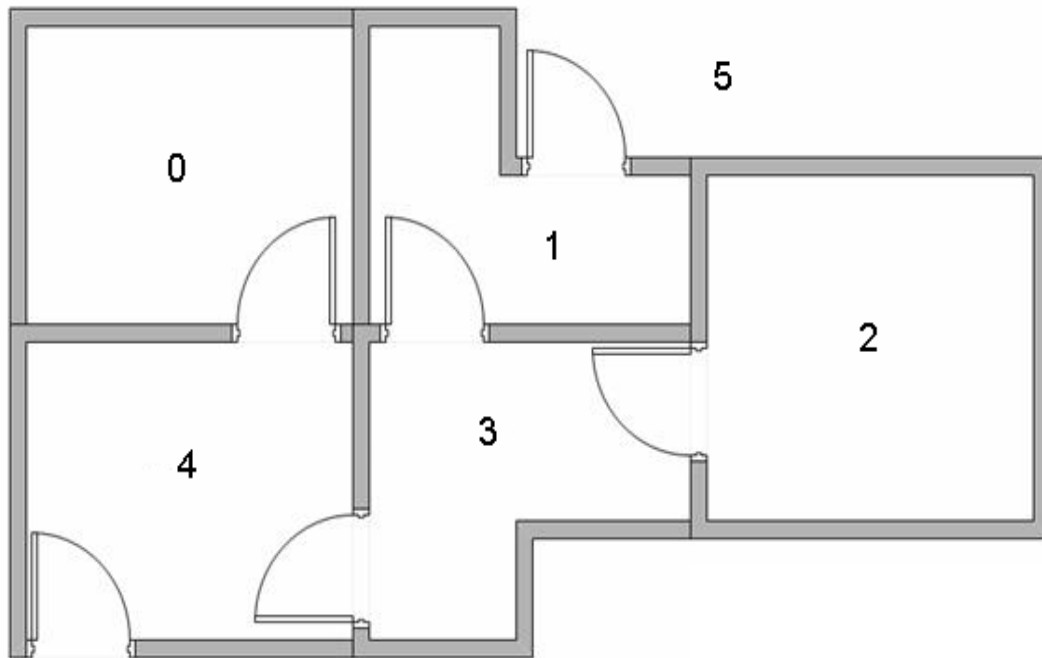
$$Q(s,a) = r(s,a) + \gamma V^*(s',a)$$

$$= r(s,a) + \gamma \max_{a'} Q(s',a')$$

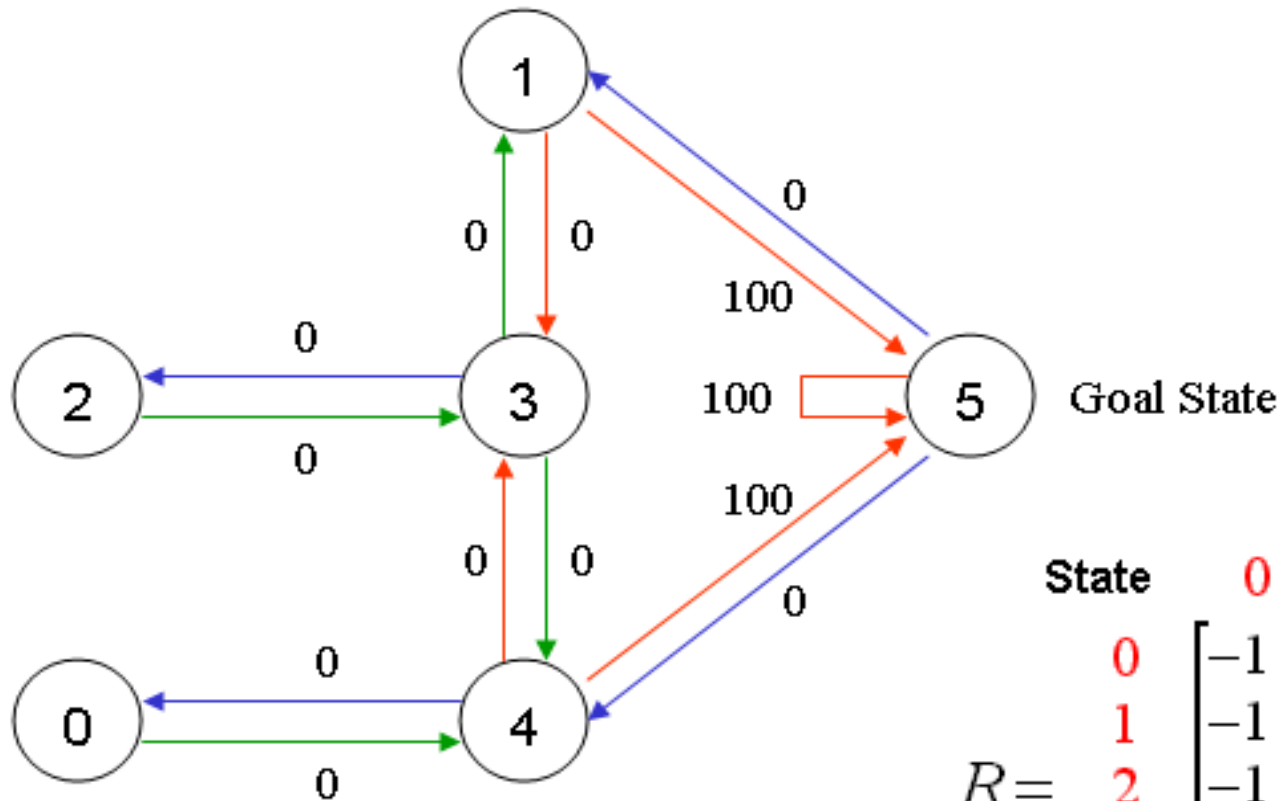
$$\pi^*(s) = \arg \max_a Q(s,a)$$

$$V^*(s) = \max_a Q(s,a)$$

Path Finding using RL



Path Finding using RL



State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

$R =$

	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Path Finding using RL (Learning stage)

$$Q = \begin{matrix} & \text{Action} \\ & 0 & 1 & 2 & 3 & 4 & 5 \\ \text{State} & \\ 0 & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ 1 & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ 2 & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ 3 & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ 4 & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ 5 & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix} \quad R = \begin{matrix} & \text{Action} \\ & 0 & 1 & 2 & 3 & 4 & 5 \\ \text{State} & \\ 0 & \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \end{bmatrix} \\ 1 & \begin{bmatrix} -1 & -1 & -1 & 0 & -1 & 100 \end{bmatrix} \\ 2 & \begin{bmatrix} -1 & -1 & -1 & 0 & -1 & -1 \end{bmatrix} \\ 3 & \begin{bmatrix} -1 & 0 & 0 & -1 & 0 & -1 \end{bmatrix} \\ 4 & \begin{bmatrix} 0 & -1 & -1 & 0 & -1 & 100 \end{bmatrix} \\ 5 & \begin{bmatrix} -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{matrix} \quad Q = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$$

$$Q(1, 5) = R(1, 5) + 0.8 * \text{Max}[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100$$

$$Q = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$Q = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

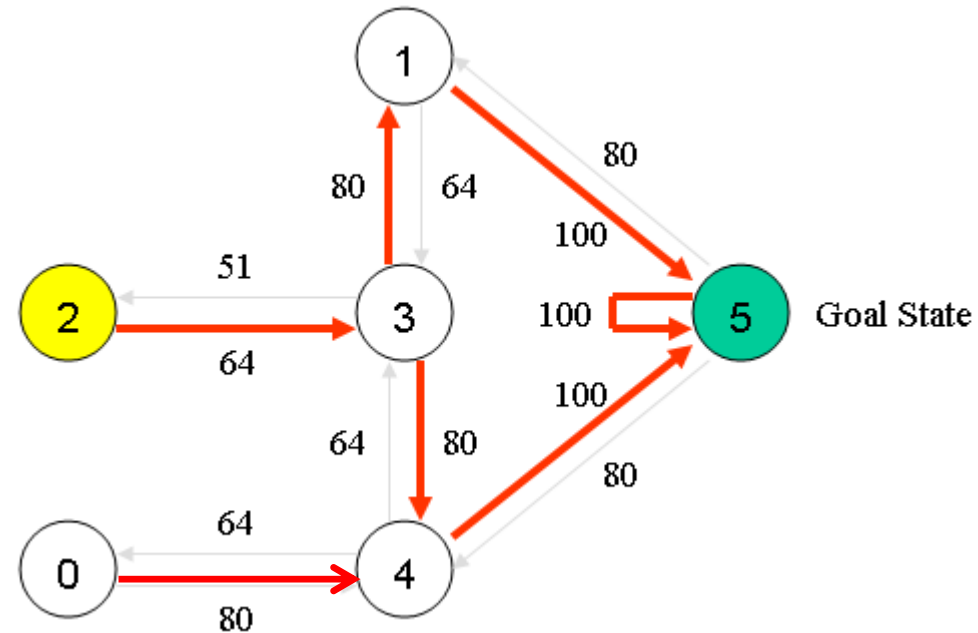
$$Q(3, 1) = R(3, 1) + 0.8 * \text{Max}[Q(1, 3), Q(1, 5)] = 0 + 0.8 * 100 = 80$$

$$Q(1, 5) = R(1, 5) + 0.8 * \text{Max}[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100$$

Path Finding using RL (Testing Stage)

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 400 & 0 \\ 0 & 0 & 0 & 320 & 0 & 500 \\ 0 & 0 & 0 & 320 & 0 & 0 \\ 0 & 400 & 256 & 0 & 400 & 0 \\ 320 & 0 & 0 & 320 & 0 & 500 \\ 0 & 400 & 0 & 0 & 400 & 500 \end{bmatrix} \end{matrix}$$

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$



```
private static final int Q_SIZE = 16;
private static final double GAMMA = 0.8;
private static final int ITERATIONS = 10;
```



```
<terminated> QLearning2 [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe (Jun 7, 2014 5:22:37 AM)
```

Figure 1 displays a 16x16 matrix of values (rows 0 to 15) and a directed graph with 8 nodes (0 to 7). The matrix values are as follows:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	40	0	0	0	63	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	252	0	0	160	0	0	0	0	0	0	0	0	0
3	0	0	201	0	0	0	0	316	0	0	0	0	0	0	0	0
4	50	0	0	0	0	0	0	0	79	0	0	0	0	0	0	0
5	0	0	0	0	0	0	160	0	0	102	0	0	0	0	0	0
6	0	0	201	0	0	128	0	0	0	0	0	0	0	0	0	0
7	0	0	0	252	0	0	0	0	0	0	396	0	0	0	0	0
8	0	0	0	0	63	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

The directed graph shows nodes 0 to 7. Node 0 is highlighted in yellow. Directed edges are labeled with weights:

- 0 → 1: 0
- 1 → 0: 0
- 0 → 4: 0
- 4 → 0: 0
- 2 → 3: 0
- 3 → 2: 0
- 2 → 6: 0
- 6 → 2: 0
- 5 → 6: 0
- 6 → 5: 0
- 7 → 6: 0
- 6 → 7: 0

Shortest routes from initial states:

1, 0, 4, 8, 9, 5, 6, 2, 3, 7, 11, 15

3, 7, 11, 15

5, 6, 2, 3, 7, 11, 15

2, 3, 7, 11, 15

4, 8, 9, 5, 6, 2, 3, 7, 11, 15

0, 4, 8, 9, 5, 6, 2, 3, 7, 11, 15

